0210. Course Schedule II

Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/course-s

Problem

- Return any valid order to take all courses, or [] if impossible.

Intuition

- The order in which Kahn's algorithm removes zero in-degree nodes is a topological order.

Brute Force

- Try permutations and validate: O(n!).

Optimal Approach

- Same as problem 207, but append each popped course to the output.
- If output length < numCourses, return [].

Worked Example

```
n = 4, [[1,0],[2,0],[3,1],[3,2]].

queue [0]       order [0]       indeg 1:0 2:0 -> queue [1,2]
pop 1           order [0,1]     indeg 3:1
pop 2           order [0,1,2]   indeg 3:0 -> queue [3]
pop 3           order [0,1,2,3]
```

Java Solution

```java
public class CourseScheduleII {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p : prerequisites) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; }
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.add(i);
        int[] order = new int[numCourses];
        int k = 0;
        while (!q.isEmpty()) {
            int c = q.poll();
            order[k++] = c;
            for (int nx : adj.get(c)) if (--indeg[nx] == 0) q.add(nx);
        }
        return k == numCourses ? order : new int[0];
    }
}
```

Complexity

- Time: O(V + E). Space: O(V + E).

Pitfalls

- Several valid orders exist; tests should validate the order rather than compare to one answer.